

Group 124 Final Project Report: TreeWaldo

Keyu Lin, Tinu Joseph

{link55,josept12}@mcmaster.ca

1 Introduction

Tree delineation is an area of research in the geospatial and forestry field that revolves around identifying individual tree crowns (tree tops) from remote sensing data. This task has a wide application of usages in both academia and industry settings that include deriving tree metrics (DBH, height, carbon storage, etc.), forest properties or environmental monitoring.

Many companies and organizations in the forestry and environmental space don't have large funding and resources compared to more private sectors. Manual methods to perform tree delineation require large amounts of time and labour that can be costly to small organizations. They can benefit from a lightweight method and model to do tree delineation. Our model aims to do produce the most accurate results while not requiring intensive training, tuning, algorithm complexity which aligns with constraints we have as undergrad students.

Much of the difficulty on tree delineation lies within the tricky nature of differentiating tree crowns due to blending with the environment or overlapping tree crowns which can serve as a challenge even to human observers.

We named our project TreeWaldo as it reminded us of the game "Who's Waldo" but instead of finding all the different Waldo characters, we're looking for all the trees.

1.1 Related Work

Existing work on tree delineation using both machine learning and non-machine learning techniques (Zhenga et al., 2023) has been done before to varying degrees of success. Traditional or non-machine learning methods employ techniques such as region growing, watershed or even local maxima for detecting tree top points or tree crowns.

WhiteboxTools (Lindsay, 2020), a common geospatial library for multiple programs and lan-

guages including QGIS, ArcGIS, Python has a tree delineation feature that detects tree top points and trees with a local maxima algorithm (Fig 1). This simple tool allows you to customize hyperparameters such as min height and min search radius for tuning to your use. However, this tool is used on a LiDAR medium which an orthophoto (RGB image orthorectified/straight down perspective) can be converted into with tools such as OpenSFM (mapillary, 2020) but introduces problems in the additional intermediary step. Local maxima techniques such as Whitebox are also susceptible to problems such as over-classification, often produces many more trees than in ground truth or misclassifications, classify non-trees as trees.

With the advent of neural network growth and popularity throughout the last decade, people turned to machine learning and deep learning approaches such as MASK R-CNN (Ball et al., 2023) and StarDist (Tong and Zhang, 2025) to perform tree delineation. CNN architectures were especially popular as they excel in handling image and remote sensing data for instance segmentation.

Many tree delineation libraries such as Detectron2 (Wu et al., 2019) library from Facebook and their pre-trained masked R-CNN model to do instance segmentation for tree crowns on RGB files. The data input, output and solution is similar to our work. However we do not plan on using a pre-trained model such as detectron and instead aim to build a malleable model from the ground up oriented towards small datasets and images.

We can use an existing object detection models as a baseline or create our own naive implementation with techniques such as traditional watershed algorithms or gradient/texture patch analysis which we will discuss more during implementation.

There are also papers on hybrid approaches (Durgut et al., 2025) with traditional features augmenting deep learning but that is



Figure 1: Visualization of WhiteBox tree detection with three.js

outside of our complexity scope and approach.

2 Dataset

There are two parts to our dataset for tree delineation. The first part is the orthorectified RGB image dataset sourced from NEON's website([National Ecological Observatory Network \(NEON\), 2025](#)) for which we will perform tree delineation on and segment out tree crowns. The second dataset contains label data in a csv format hosted on Zenodo([Weinstein et al., 2020](#)) for the coordinates of reference tree crowns bounding boxes so that we can train and evaluate how effective our methods are. There were multiple label datasets for different plots and years of the NEON repository. Datasets were only downloaded and processed for the ABBY_2019 site which contains 1048575 trees and 108 images pre-processing.

The image files from NEON are tif files contain multi-band(also considered layers) information across it's 10000 x 10000 pixel grid(0.1m resolution) for each tif file. Tif files are commonly used in geospatial areas as they store geographical information such as RGB, height, classification, elevation,

etc in bands that geospatial tools and libraries can read in addition to positional data. This is important as it allows our dataset to encode more information and features in a with just one image as our input. Some of these photos are warped which is common with photogrammetry. We will assume that images only contain RBG data as additional other bands are seen as optional and would be unrealistic to assume.

The NEON Tree Crowns Labels CSV dataset contains a multitude of features including bounding box coordinates for delineated trees, id, site, etc. We will primarily use this dataset as truth labels we expect to get from an output as well as using of the other features for evaluation such as area for possible overlap count.

We can eliminate some features from the CSV file as we are not interesting in predicting height metrics, not using the shape files(as image identification can be done with geo index alone). ID field is optional depending on if we want to locate specific data points later on that may be useful for debugging source of errors and inaccuracies.

2.1 Preprocessing

Our original NEON image dataset spanned hundreds of GB for the ABBY plot alone which was not feasible to process given our constraints on time and resources. To obtain 1000+ data points and images for model training without having a big processing size, we decided to segment the image files into a 10 non-overlapping cells to produce 100 images which still contain multiples of trees within each image to shrink our overall input size but be able to use the dataset with more instances (Fig 2). The partitioning was done with a separate script and the resulting partition images were saved locally. One issue that does arise from this partitioning preprocess is how to handle trees whose boundaries lie on the edge of a partition and how do we know which tree belongs to one partition if we treat them as separate images.

We augmented the data during preprocessing and added a column for partition ID corresponding to which partition the tree resides in. Trees that belong to two or more partitions were marked with a partition ID of -1 which can either be tossed out and deleted from our input data either in another script or when loaded into our main program. There is an alternative approach to handle partitioning and shrinking image plot sizes where partitions contained overlap such that every tree is fully contained at least within one partition but this creates duplication problems and you still will get trees that have one half of a tree in two partitions. It's a known fact in forest analysis especially for forest inventory that photogrammetry data should contain some overlap, with the overlap percentage up for debate(usually it's the more the better). In this paper(Goodbody et al., 2021) they benchmarked different overlap values on their forest inventory application impact and found that past 60% overlap found no significant different but at lower values it does make a difference. However, we don't believe this impact will affect our tree delineation task as much as it would for forest inventory measurements or application task such as detecting basal area and above ground biomass as segmenting tree crowns is a more individual task that can work with a relatively smaller neighborhood. For simplicity in logic and abundance of samples(non-overlap only destroys about < 10%), we decided to go with the former approach.

Overall our dataset will contain 10 full sized RGB images cut down into 1000 partitioned RGB



Figure 2: Partitioning Process with No Overlap

images and over 1 million annotated tree bounding boxes(of which we will only use 252 778). Here is a list of each image and the number of trees contained within before partitioning:

1. 551000_5069000: 29590 trees
2. 553000_5064000: 37654 trees
3. 553000_5062000: 24940 trees
4. 551000_5065000: 27336 trees
5. 554000_5068000: 17955 trees
6. 554000_5063000: 26014 trees
7. 551000_5071000: 14580 trees
8. 552000_5063000: 18255 trees
9. 551000_5062000: 33238 trees
10. 554000_5072000: 23084 trees

3 Features

As our model will take in inputs from two different sources, one primarily provides features used for training while the other contains labels for loss comparison and evaluation.

Fields used in preprocessing

1. geo_index(csv)
2. RGB data(images)
3. Texture Data(images)
4. partition_id(csv)

Partition ID is to let us know which partition image we are working with to pass in the correct raster/image and normalize the RGB data. The

RGB data technically does not have to be normalized during preprocessing as they are the only input features that all have the same scale 0-255 but normalized inputs to a CNN or other models can be beneficial to learning so we apply normalization. Geo_index field directs to the correct plot and set of partitioned files as we plan on training with more than 100 images, up to 1000 which will span multiple geo_plots of the overall ABBY site. Geo index and partition ID are scalar values we read for one instance but RGB data will be read and stored as a NP array. Texture data will be a self created field when processing the RGB images to give us a texture layer we can add on as a feature to pass in the model. This will hopefully allows us to discern and learn the texture of a tree vs background objects which is useful because if you look at the dataset, most of the images are green with fields, trees, shrubbery. Likewise we apply normalization to texture values by a constant 255.

Input features

1. RGB data
2. Texture

RGB and texture data will be sent in as input to the model as it tries to predict trees. This is a common assumption for input RGB files to only have RGB layers and texture values will be derived from the patches we view. At this stage the input tensor has four dimensions for the input channel(R, G, B, Texture).

Evaluation

1. Bounding Box(BBox) coordinates
2. Predicted Bounding Box coordinates
3. partition_id

The above fields are used for evaluation metrics. XY coordinates will be saved to a CSV after model training and prediction and denote the geoposition of predicted tree bounding boxes which are matched with reference tree geopositions. There needs to be some unit and coordinate system conversion as the ground truth data is in UTM coordinates while predicted processed coordinates will be in pixels. We can determine how many matches there are between reference and prediction trees and calculate evaluation metrics which will further gone into detail during the evaluation and error analysis section.

We tried just different input features with just RGB and UTM coordinates but found that learning coordinates made no sense for detecting trees and RGB information by itself may not be enough hence the addition of a texture layer created from RGB bands.

4 Implementation

Our model uses a supervised learning approach to detect and outline tree crowns from aerial RGB images. The implementation connects the geospatial information in the ABBY dataset with image-based learning using a U-Net convolutional neural network (CNN). Each image tile corresponds to a fixed geographic area identified by geo_index and partition_id. These identifiers make sure that the images and their labeled crowns line up correctly during training and evaluation. To give the network more information, we add a fourth texture channel created from the Laplacian of the grayscale image.

Our initial prototype used a patch approach that works on a set 48×48 pixel image patches. The model has three main convolutional layers with ReLU activations, followed by average pooling and a fully connected layer that outputs a single probability between 0 and 1. This represents how likely it is that the patch contains a tree crown.

We augmented the data to create negative and positive samples from our dataset by iterating on random windows/patches on an image. These samples were then appended and returned to be split into training and testing sets. This lets us learn the features of a tree given a patch on the image. We are able to make the instance segmentation problem into a binary classification by iterating over patches and doing tree crown classification.

Since our approach to tree delineation is a binary classification task, we used Binary Cross-Entropy Loss (BCEWithLogitsLoss) with Adam optimizer

Our final prototype switched from a patch-based method to a well known image segmentation architecture: U-Net. U-Net CNNs have encoder and decoder capabilities that allows us to capture pixel level probabilities of belonging to a tree crown rather than iterating over patches. Similarly to our patch prototype, we also augment the data and create a texture layer to add onto our RGB bands for four input channels into our model. The next step to our pre-processing included another similar step where we converted the existing ground truth boxes into mask regions where we expect to predict tree

crowns and return an image tensor, mask tensor for each tile partition under a TreeSegTileDataset object.

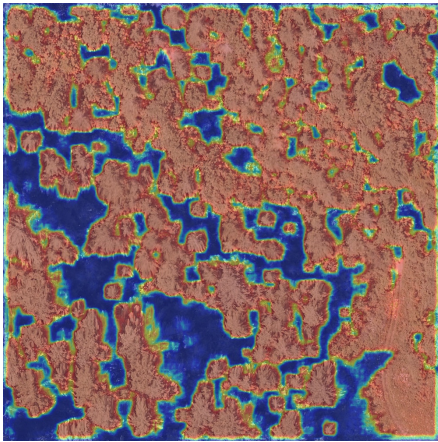


Figure 3: Probability map for tree crown regions

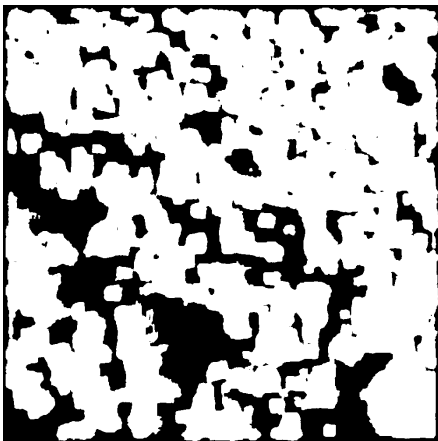


Figure 4: Mask that the model uses to decide bounding boxes

Our U-Net was a simple implementation with 3 encoding layers, 2 decoding layers, and a convolutional block consisting of a convolution layer following by batch normalization, ReLU activation and another convolution, batch, ReLU. For our loss function and criterion, we built on top of binary cross entropy loss with dice functionality to alleviate the effect of majority tree cover in our training data.

After splitting our data into train and validation set, we trained the model for 5 Epochs (anything more took much too long) and evaluated the validation loss using Intersection over Reference (IoR) methods between predicted tree bounding boxes and ground truth ones.

The last step after our model training is to post-process the compute logits using our model weights

to create a probability and mask map that displays regions of expected tree crowns. After the probability and mask layers are created, we apply watershed algorithm and seed the region with possible tree crown centers and generate bounding boxes. Then we apply various filters such as min area size, max area size, non-maximal suppression (NMS), probability threshold, min_distance to collapse and get rid of excess bounding boxes and get an accurate span of boxes across the masked regions. When NMS is applied we weight bigger areas in addition to their tree crown probability to decide which tree crown of the pairwise comparison gets collapsed. These final boxes become our final crown detections which we make drawing boxes on and append to our results csv.

Many different parameters/hyperparameters were considered, a few will be shown in the appendix section and their resulting boxes drawn.

As a baseline for comparison, we considered a simple threshold-based approach using the average pixel intensity and vegetation color to estimate canopies.

The dataset is split into 80% training and 20% validation tiles. During training, the model is ran for 10 epochs. During validation, we check accuracy by comparing the predicted tree probabilities with the known labels.

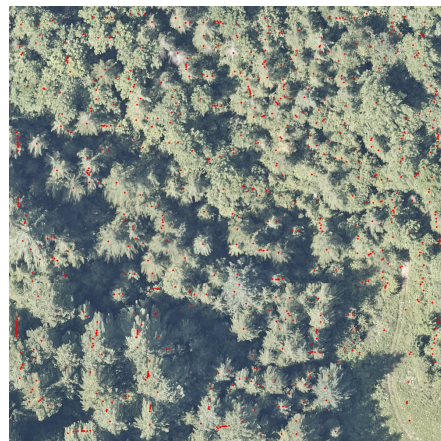


Figure 5: Bounding box center "seeds" that will become bounding boxes after watershed algorithm

The project is implemented entirely in Python using PyTorch for deep learning, OpenCV for image processing, NumPy and Pandas for data handling, and Rasterio for geospatial alignment. Rasterio ensures that every tile maintains its real-world coordinates, while OpenCV handles the patch extraction and visual output. The code also saves

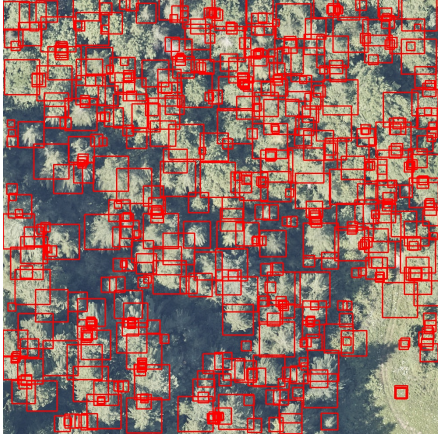


Figure 6: Predicted bounding boxes drawn with region centers

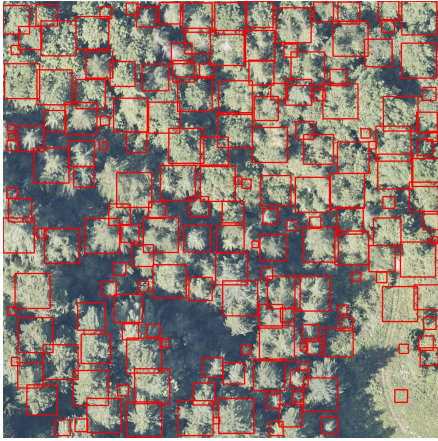


Figure 7: Bounding boxes after NMS and other filters are applied

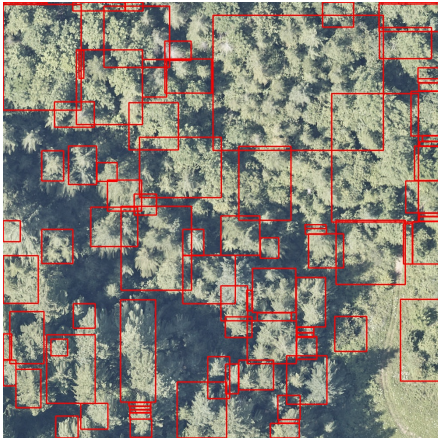


Figure 8: Bounding boxes from Patch based model, from an eye test, we clearly see that many boxes span areas that should encompass multiple tree crowns, a limitation that led to the switch to U-Net architecture

debug images such as probability heatmaps, binary masks, and bounding-box visualizations pre- and post-NMS so that results can be inspected visually.

File paths are dynamically pulled from the same directory and parameters are configurable, making it easy to scale up and include more ABBY plots in future training runs.

5 Evaluation

Our evaluation pipeline is built around comparing predicted bounding boxes against a reference set of manually delineated tree crowns. Our dataset consists of high-resolution orthographic tiles from the ABBY site, and for each tile we generate detection outputs using both the baseline model and our prototype deep learning model.

Because our goal is not classification but instance-level object localization, we do not use a typical train/validation/test split based on class distributions. Instead, each tile acts as a spatially distinct sample. We train on a 80% of annotated tiles, validate on the remaining 20% held-out subset during training, and reserve the remaining tiles as the test set used for all quantitative reporting. The label distribution is effectively determined by the density of trees within each tile; some regions contain sparse crowns while others are highly dense, which we later show impacts per-partition recall. Compared to our Progress report we are now running our tests on the full scope of data instead of a single ABBY image partitioned so the Baseline values will be different to represent this change.

The core of our evaluation is a matching algorithm that pairs each reference bounding box with the prediction that has the highest Intersection-over-Reference (IoR). If the IoR is at least 50%, we treat the pair as a match (true positive). A reference tree with no matched prediction is a false negative, and predicted boxes that were not matched are counted as false positives. We do not compute true negatives, because the spatial domain is continuous and unbounded, you could define arbitrarily many “empty” regions where no tree exists. In reference to Table 1

Model	Precision	Recall	F1
Baseline	0.3022	0.3221	0.3118
U-Net	0.6085	0.6524	0.6296

Table 1: Evaluation metrics for the baseline and U-net prototype model

Beyond the standard TP/FP/FN counting, our evaluation now incorporates several additional metrics that were not part of the progress report stage.

First, we compute precision, recall, and F1 score at the full dataset level to summarize overall detection performance.

However, the new version of our evaluation pipeline expands on this by introducing two quality-focused analyses that directly address shortcomings in our earlier reporting:

1. **IoR Quality Distribution:** Instead of only counting whether a match exceeds the IoR threshold, we now measure how good those matches are. For every true positive, we record the actual IoR value and generate a distribution plot. This lets us distinguish between “barely acceptable” detections and genuinely accurate crown overlaps. A model that produces many low-IoR TPs may achieve the same recall as one with tight, high-IoR overlaps, but the qualitative difference matters for downstream ecological tasks. This is an insight that was completely missing from the progress report, where we only looked at binary $\text{IoR} \geq 0.5$ matches. The threshold for a good IoR is 75%.
2. **Partition level Density vs Recall Correlation:** Since each partition varies drastically in the number of trees per tile, we now evaluate how tree density impacts model performance. For each partition we compute TP, FP, FN, and local recall, and then correlate recall with ground-truth density. This gives us direct evidence about whether the model struggles in dense canopies or performs consistently across sparsity levels. This density aware evaluation highlights structural weaknesses in the model that simple global metrics cannot reveal. Again, this was a major limitation of our earlier evaluation, where results came from a single ABBY image and we treated all tiles as though they were equally difficult.

We now generate supplemental plots for this new data and even the old data that was in the previous report like a confusion matrix which could have been made in the progress report. We now additionally have a scatter plot with the density vs. recall with a line calculated by the Pearson correlation and an IoR histogram to visualize the quality of detections. These will all be discussed in the Error Analysis section and the graphs will be provided there. Lastly since we now keep track of the per partition values for TP/FP/FN we can also

show how it looks between partitions on a chart so it is easily visible how differently the model performed on all of them, a straight line would demonstrate that our model is very consistent no matter the type of image it is given.

6 Progress

In the progress report, we laid out a plan focused on improving our original CNN-based detector, expanding the dataset, strengthening regularization, and potentially exploring a more sophisticated architecture such as an R-CNN style pipeline. We partially followed this plan, but we also shifted direction once we realized the limitations of our original approach.

We did follow through on the main idea of developing a deeper, more capable model, but instead of incrementally expanding our old CNN, we pivoted to implementing a full U-Net segmentation model. This change came from recognizing that the bounding-box-only detector we built earlier simply was not expressive enough for the structure of tree crowns. Tree crowns vary wildly in shape, touch each other, and blend into backgrounds, so treating them as category level objects with simple rectangular bounding boxes was fundamentally limiting. The U-Net approach allowed us to model crown regions at the pixel level and then derive boxes from segmented instances, which aligned more closely with the real geometry of the trees and gave us far more control over post-processing (distance transforms, watershed separation, NMS, texture filtering, etc.)

Another part of our plan that changed was our intention to build a new baseline model. Early on, we expected to compare our improved method against both a handcrafted baseline and a lightweight CNN detector. But as the project progressed, the CNN prototype wasn't pursued. It wasn't worth enough to justify further tuning, and keeping it in the evaluation would not have added meaningful insight. At the same time, creating a brand-new baseline would have been a poor use of time, since any simple model we could build would still underperform and distract from developing the segmentation approach that actually pushed the project forward. So instead of spending effort on a weaker model we knew we wouldn't use, we focused fully on the U-Net model and treated it as our primary learning-based method for the final evaluation.

We did follow through on expanding the dataset.

Instead of working with a single ABBY tile, the segmentation model was trained and validated on all 10 images (1000 partitions). The training/validation split, dataset normalization, texture channel computation, and full-tile augmentation pipeline were all implemented as planned, and the larger dataset clearly helped with learning more generalizable canopy patterns.

Finally, we executed on our plan to upgrade our evaluation pipeline. In addition to re-running the detection metrics on the full dataset, we added several analyses that were proposed in the progress report: IoR quality distributions, a partition level performance summary, and density–recall correlation. These additions provided additional insight that wasn’t in our earlier evaluation with just precision, recall and F1.

7 Error Analysis

To understand how both our baseline detector and U-Net segmentation model behave in practice, we ran a detailed error analysis using confusion-matrix results, IoR quality scores, and per-partition metrics. We began by matching each ground-truth tree crown with the prediction that gives the highest Intersection-over-Reference (IoR). Matches with $\text{IoR} \geq 0.5$ count as true positives, while all unmatched predictions and unmatched trees become false positives and false negatives. This gives us a clear picture of how often each model detects a crown, misses it, or fires incorrectly. We then measured the quality of correct detections by looking at the full IoR distribution. Finally, because the ABBY site contains areas with different numbers of trees, we computed precision, recall, and F1 for each image partition and examined how model performance changes depending on how many crowns are present.

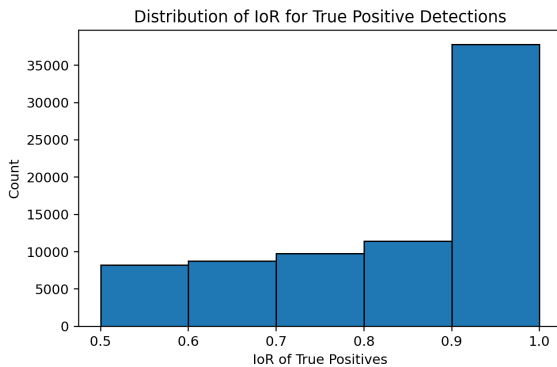


Figure 9: Baseline Histogram

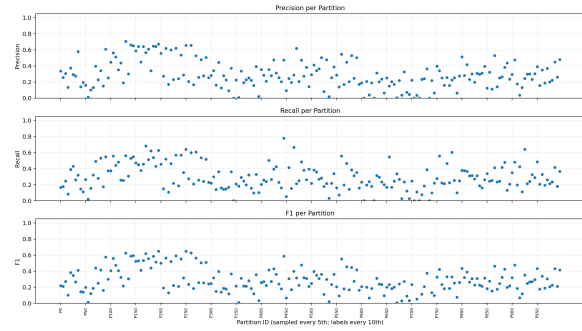


Figure 10: Baseline Per-partition Evaluation

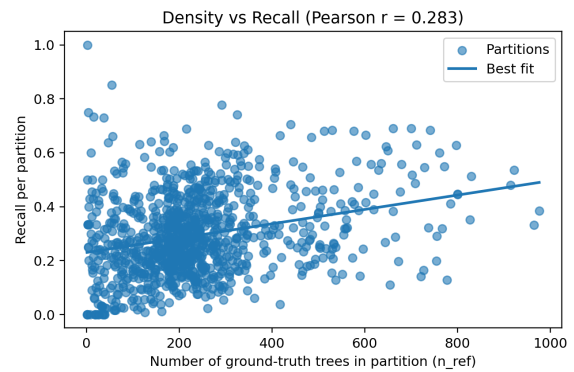


Figure 11: Baseline Density and Recall

Reduced Confusion Matrix	
Actual Tree	Predicted Tree
	No Prediction
Yes	75964
No	159896
No GT Tree	175387
	N/A

Figure 12: Baseline Confusion Matrix

The baseline model shows clear limitations. It detects 75,964 correct crowns, but produces 175,387 false positives and 159,896 false negatives. This results in 30.22% precision, 32.21% recall, and a 31.18% F1 score. Although the baseline performs poorly overall, the true positives it does identify are usually well-aligned with ground truth:

the average IoR is 0.8486, and 71.40% of its correct detections have $\text{IoR} \geq 0.75$. This means that when the baseline is confident, it tends to place tight bounding boxes, but these moments are rare.

The biggest source of error comes from the model's handcrafted ExG and texture-based rules. These often confuse grassy areas, shadows, and textured ground patches for tree crowns, creating large amounts of false positives. At the same time, the model misses many real crowns, especially in parts of the image where trees overlap, where the canopy is darker, or where crowns are small. We also looked at whether the baseline performs better in places with more trees. The correlation between tree density and recall is weak (Pearson $r \approx 0.28$), meaning the baseline struggles somewhat uniformly no matter how many crowns are present.

The U-Net shows a very different error profile and is much more reliable overall. It correctly detects 153,870 crowns, cutting false negatives nearly in half (81,990) and substantially reducing false positives to 99,015. The resulting precision (60.85%), recall (65.24%), and F1 score (62.96%) are all roughly double those of the baseline. This demonstrates that the U-Net not only makes far fewer mistakes but also covers the canopy much more consistently.

Reduced Confusion Matrix

Actual Tree	153870	81990
No GT Tree	99015	N/A
	Predicted Tree	No Prediction

Figure 13: U-net Confusion Matrix

We also checked the quality of each true positive. The U-Net's average IoR is 0.8382, with 67.40% of correct detections scoring $\text{IoR} \geq 0.75$. This is slightly lower than the baseline's best matches but across a much larger number of true positives.

The slight drop in IoR comes from the watershed step used to convert the segmentation mask into instance-level crowns. In dense regions this process sometimes over-segments a single tree into multiple predictions, causing weaker box overlap even though the model correctly identified the tree's location.

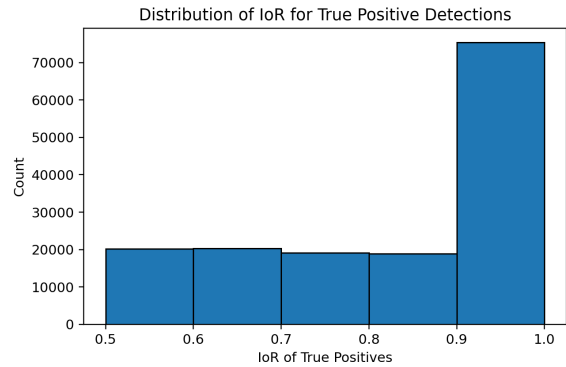


Figure 14: U-net Histogram

In terms of how performance changes with tree density, the U-Net improves as the number of crowns increases (Pearson $r \approx 0.37$). This makes sense, as the segmentation mask becomes more confident when it sees large, continuous canopy regions. The model's main errors include missing very small trees, splitting a single crown into multiple pieces, and occasionally mistaking bright grassy patches for trees.

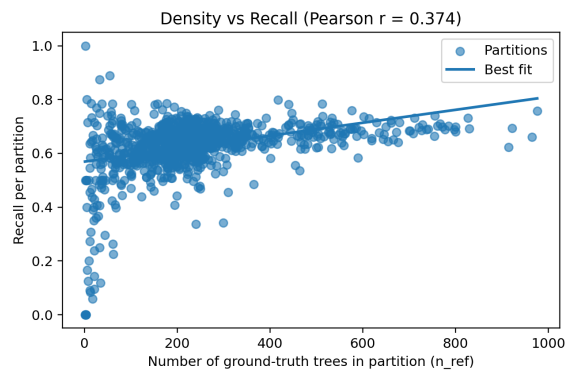


Figure 15: U-net Density and Recall

When we compare the two models directly, the U-Net outperforms the baseline in almost every meaningful way. The baseline occasionally produces very accurate bounding boxes but does so inconsistently and often in the wrong locations. The U-Net, on the other hand, provides far more balanced performance, capturing most trees while keeping false positives manageable. Both models

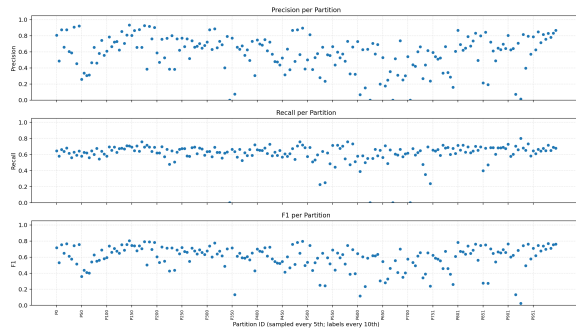


Figure 16: U-net Baseline Per-partition Evaluation

struggle with extremely small crowns and with areas of deep shadows, but these issues are much more pronounced in the baseline.

8 Future Work and Notes

If we were to continue improving the system, a few possible ideas for improvement include

- apply reseeding to high probability regions(or white mask regions) after NMS to fill in gaps better until iterations are done
- Create a distance-area suppression step where small boxes get absorbed, not destroyed into nearby boxes maybe theirs bigger(because it's right that there should a only be 1 and not 2 crowns, but the crown bounding box size is wrong)
- Incorporating additional input channels such as NDVI or height data could also help the model tell grass apart from tree canopy
- Implement negative mining, this was in the initial patch network but after I reviewed it, there are some partitions that don't have or barely any negative samples which makes it harder to implement
- Try out gradient masks instead of binary for the initial mask building before training

Adding a Laplacian filter for ground didn't help with removing detecting grass as trees. Much of the work and improvements for this task came outside of the model and in the data pre-processing(texture addition)/post-processing(watershed detection) which shows it's not just about the model but the data around it.

9 Team Contributions

Keyu and Tinu worked on the report, final CNN model. Tinu worked on the evaluation and baseline model. Keyu worked on the final partitioning script. There was input and feedback on all tasks so it wasn't fully individual at any point. Non-ML Diagrams were made by Keyu.

References

- James G. C. Ball, Sebastian H. M. Hickman, Tobias D. Jackson, Xian Jing Koay, James Hirst, William Jay, Matthew Archer, Méline Aubry-Kientz, Grégoire Vincent, and David A. Coomes. 2023. [Accurate delineation of individual tree crowns in tropical forests from aerial rgb imagery using mask r-cnn](#). *Remote Sensing in Ecology and Conservation*, 9(5):641–655.
- Ozan Durgut, Beril Kallfelz-Sirmacek, and Cem Unsalan. 2025. [Integrating traditional and deep learning methods to detect tree crowns in satellite images](#). *Preprint*, arXiv:2507.01502.
- Tristan R.H. Goodbody, Joanne C. White, Nicholas C. Coops, and Antoine LeBoeuf. 2021. [Benchmarking acquisition parameters for digital aerial photogrammetric data for forest inventory applications: Impacts of image overlap and resolution](#). *Remote Sensing of Environment*, 265:112677.
- Dr. John Lindsay. 2020. [Whiteboxtools user manual](#). Last accessed 9 November 2025.
- mapillary. 2020. [Opensfm: open-source structure-from-motion library](#). GitHub repository.
- National Ecological Observatory Network (NEON). 2025. [High-resolution orthorectified camera imagery mosaic \(dp3.30010.001\)](#).
- Fei Tong and Yun Zhang. 2025. [Individual tree crown delineation in high resolution aerial rgb imagery using stardist-based model](#). *Remote Sensing of Environment*, 319:114618.
- Ben Weinstein, Sergio Marconi, Alina Zare, Stephanie Bohlman, Sarah Graves, Aditya Singh, and Ethan White. 2020. [Neon tree crowns dataset](#). Dataset on Zenodo.
- Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. 2019. Detectron2. <https://github.com/facebookresearch/detectron2>.
- Juepeng Zhenga, Weijia Lie Shuai Yuana, Haohuan Fub, and Le Yu. 2023. [A review of individual tree crown detection and delineation from optical remote sensing images](#). *IEEE Geoscience and Remote Sensing Magazine*, arXiv:2310.13481. Version 1.

A Appendix

Here are the different detection(and baseline) algorithms we tried and their visualized bounding boxes and scores.

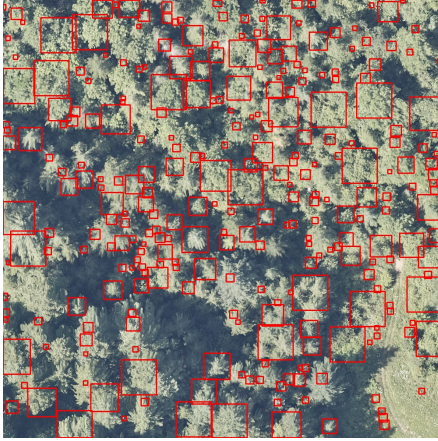


Figure 17: Pre-Area Score Model

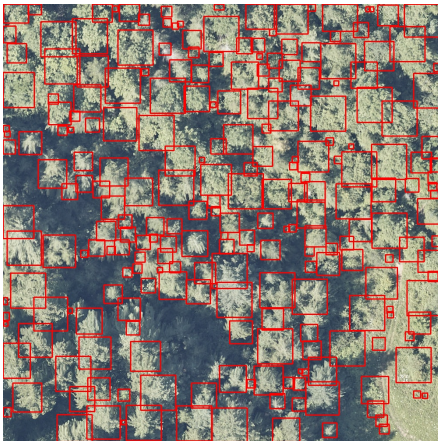


Figure 18: Post-Area Score Model

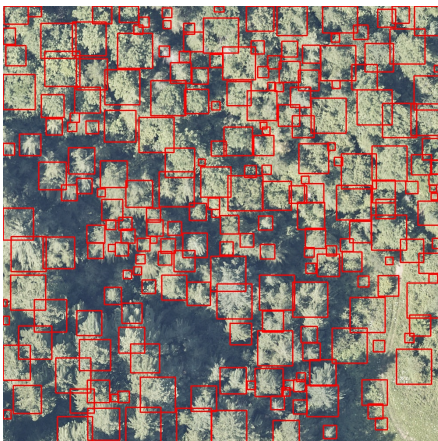


Figure 19: Post-Erosion Model

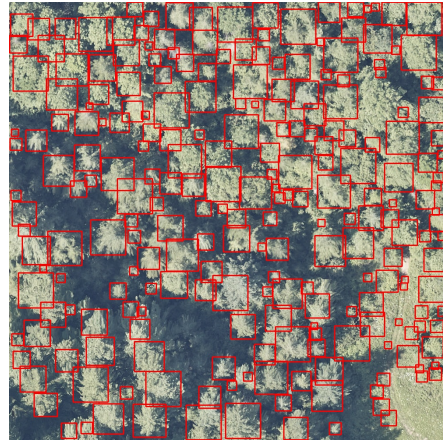


Figure 20: Positive Weight = 1

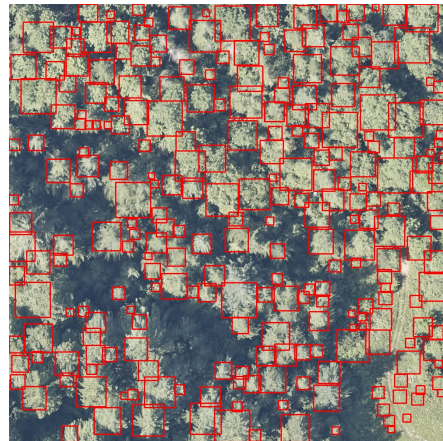


Figure 21: Positive Weight = 3

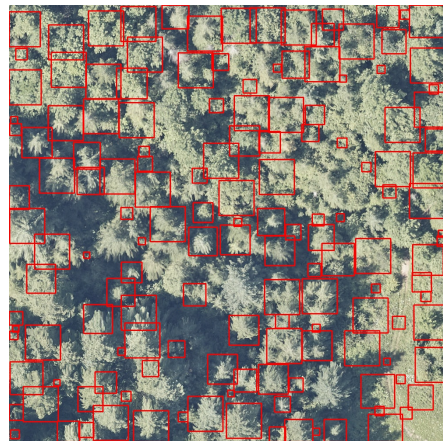


Figure 22: Positive Weight = 5